

AEGIS-EVO

*Technical Case Study: Autonomous Continuous Software Evolution
and Sandboxed Security*

RESEARCH PROTOTYPE

FIDES SECURITY LATTICE

gVisor / Firecracker

NOT PRODUCTION-READY

Python 3.11

AMD EPYC 7002

Ubuntu 24.04 LTS

128 GiB DDR4

Project Aegis-Evo is a research-driven prototype and secure systems-integration architectural blueprint. It unites a **FIDES information-flow security lattice**, an **AST-guided structural dependency mapper**, and an **ephemeral hardware-isolated sandbox** into a secure, practical multi-agent runtime for autonomous software engineering — addressing both codebase-scale evolution and cross-domain indirect prompt injection (XPIA) threats.

Research Prototype • Not for Production Use

0 Contents

1	Summary	3
2	Context & Constraints	3
2.1	Operational Security Constraints	3
3	Problem Statement: The Codebase-Scale Evolution Barrier	4
3.1	Structural Complexity and Code Tangling	4
3.2	Dynamic Execution Hijacking — The Escape Kill Chain	5
4	Technical Design & Component Architecture	5
4.1	System Architecture Overview	6
4.2	FIDES Information-Flow Security Lattice	6
4.2.1	Security Model	6
4.2.2	Permissible Information Flow Rule	6
4.2.3	Label Assignments in Aegis-Evo	6
4.2.4	The Blocking Guarantee	7
5	High-Signal Implementation Excerpts	7
5.1	Phase 1 — Ephemeral Sandbox Configuration	7
5.2	Phase 2 — Structural AST Traversal	8
5.3	Phase 3 — FIDES Lattice Interceptor Middleware	8
5.4	Phase 4 — Dynamic Performance & Efficacy Evaluation	9
6	Pilot Evaluation & Results	10
6.1	Experimental Methodology	10
6.2	Evaluated Configurations	10
6.3	Results	11
6.4	Analysis of Runtime Telemetry	11
6.4.1	Configuration 1 — Baseline (Unsecured)	11
6.4.2	Configuration 2 — Sandbox Only (gVisor)	11
6.4.3	Configuration 3 — Project Aegis-Evo	12
6.5	Pilot Scope & Statistical Caveats	12
7	Lessons Learned & Architectural Limitations	12
7.1	Structural Separations Overcome Prompt Engineering Gaps	12
7.2	Heuristic Taint-Tracking vs. Compiler-Level IFC	12
7.3	Virtualization Latency & Pooling Strategies	12
7.4	Code Comprehension via AST Graphs	13
8	Project Engineering Resources	13

1 Summary

In modern AI engineering, deploying autonomous agents to evolve large-scale, multi-file codebases remains an open challenge. Contemporary large language models (LLMs) achieve high success rates on isolated, single-file bug patches — but their efficiency **degrades significantly** for continuous, repository-scale evolution across interconnected subsystems.

This capability barrier is compounded by severe runtime security concerns. Granting an autonomous agent authority to write, compile, and execute code dynamically introduces a highly critical attack surface. If the agent ingests external code, untrusted libraries, or maliciously crafted issue descriptions, it is vulnerable to **Cross-domain Indirect Prompt Injection (XPJA)**.

Risk	Impact
Decision loop hijacking	Agent executes attacker-controlled logic
Arbitrary command execution	Shell access from within the agent runtime
Host credential exfiltration	Service tokens, cloud keys compromised
Container escape	Full host filesystem and orchestrator access

IMPORTANT

Aegis-Evo addresses this by cohesively uniting three components into a secure, practical multi-agent runtime:

- **FIDES** — Dynamic information-flow security lattice: mathematically modeled taint-tracking middleware
- **AST Mapper** — Structural, syntax-aware codebase dependency graph navigation
- **Ephemeral Sandbox** — gVisor / Firecracker hardware-isolated virtual runtime

2 Context & Constraints

Deploying autonomous developer agents (automated CI/CD patchers, pull-request reviewers) introduces a threat model that **traditional AppSec tooling cannot address**:

- **Static scanners** operate out-of-band and pre-deployment — relying on known CVE signatures
- **Autonomous agents** generate code dynamically, creating *emergent execution behaviors*
- **The core problem**: the agent processes both natural language (data) and executable code in the same context window — making it inherently vulnerable to cross-domain execution hijacking
- A malicious actor can embed obfuscated commands in **code comments, database records, or markup files** — when parsed by the agent, these bypass semantic system prompts and trigger unauthorized tool usage

2.1 Operational Security Constraints

Aegis-Evo was designed under four strict constraints:

CONSTRAINT 1 — Host Kernel Isolation

Compilation and test execution must run in environments that prevent direct access to the host OS kernel, neutralizing container breakout vectors.

CONSTRAINT 2 — Zero-Trust Network Egress

Outbound network traffic from the execution environment must be locked down, restricting communication to allowlisted package registries (e.g., PyPI) to prevent dynamic data exfiltration.

CONSTRAINT 3 — Information Flow Gating

Low-integrity data ingested from untrusted external repositories must be restricted from influencing high-integrity system calls, filesystem writes, or command executions — independent of the planner’s semantic state.

CONSTRAINT 4 — Structural Dependency Mapping

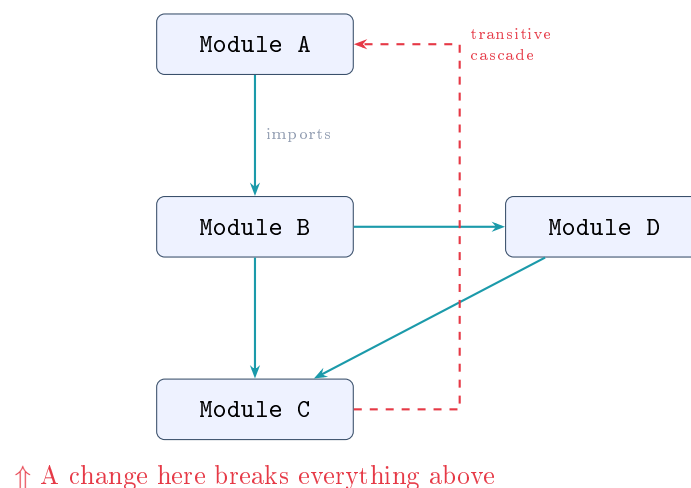
Code navigation tools must operate on structural, syntax-aware graphs rather than naive string or vector searches, minimizing the risk of “code tangling” where cycles of dependency overwhelm the agent’s planning phase.

3 Problem Statement: The Codebase-Scale Evolution Barrier

Autonomous software engineering performance degrades significantly when transitioning from isolated, single-file bug repairs to **continuous, multi-file software evolution**. Two primary challenges drive this:

3.1 Structural Complexity and Code Tangling

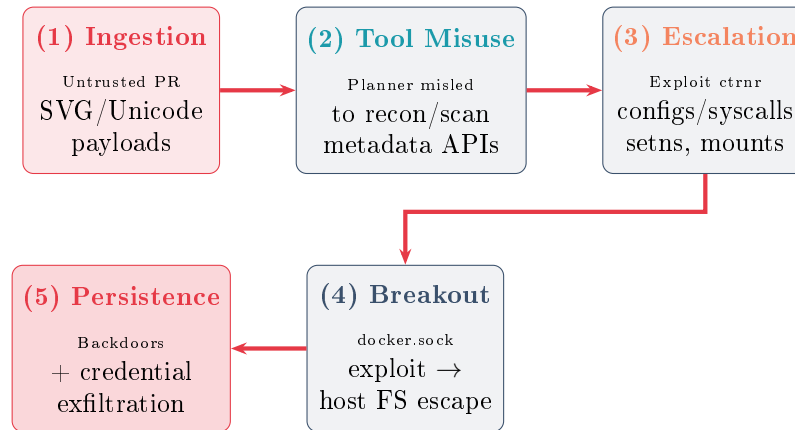
In large-scale codebases, import paths and system dependencies are highly interconnected — creating overlapping dependency cycles called “**code tangling**”:



Why naive search fails: Standard RAG / vector indexing does not capture these structural relations. The agent’s planner makes incorrect assumptions, exhausts its context window, and fails the task.

3.2 Dynamic Execution Hijacking — The Escape Kill Chain

To evolve code, an agent must compile files and execute verification suites. A codebase containing an indirect prompt injection can be used to execute arbitrary code. This vulnerability forms a **five-stage system breakout kill chain**:



Stage	Mechanism	System-Level Impact	Detection Signal
(1) Ingestion	Untrusted PR with obfuscated SVG payloads or Unicode formatting fetched	Malicious instructions enter the agent's context window	Token sequence anomalies in prompt buffers
(2) Tool Misuse	Injected prompts manipulate the planner to misuse system-level tools	Agent queries cloud metadata endpoints, performs file recon	Anomalous reads to service account tokens or AWS metadata endpoints
(3) Escalation	Exploit container configs or syscall vulnerabilities	Privilege escalation within local runtime	Execution of <code>setns</code> syscalls or unauthorized filesystem mounts
(4) Breakout	Exploit shared-kernel sockets (e.g. <code>/var/run/docker.sock</code>)	Container escape to host filesystem or orchestrator control plane	Spawned shell processes from the agent run-harness
(5) Persistence	Install persistent backdoors, exfiltrate harvested credentials	Permanent system compromise and data breach	Large outbound payloads to unallowlisted external IPs

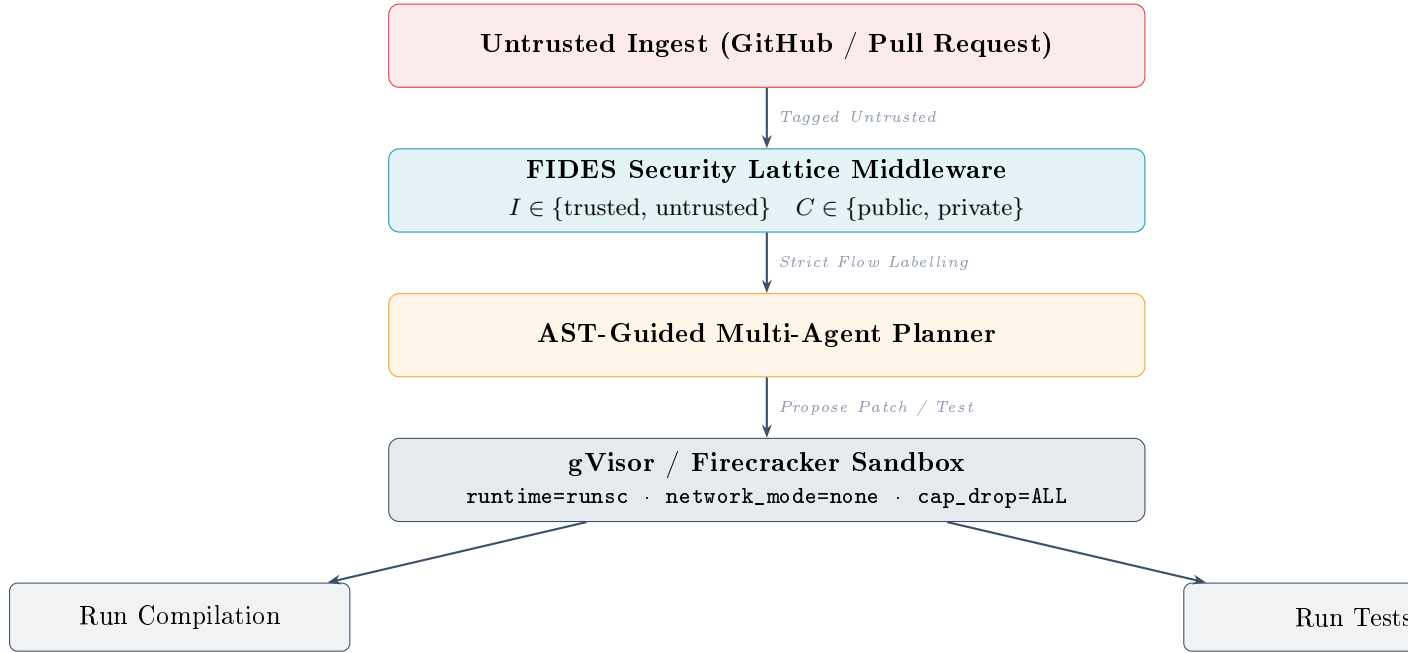
WARNING

Traditional security controls are **blind** to this kill chain. Malicious instructions are embedded within standard data paths (source code comments, issue descriptions). Static image scanning cannot detect the emergent behavior of a compromised agent.

4 Technical Design & Component Architecture

Aegis-Evo isolates raw data from execution pathways through a **multi-tiered trust boundary**. The focus is on *systems-integration innovation* — not novel cryptographic primitives.

4.1 System Architecture Overview



4.2 FIDES Information-Flow Security Lattice

FIDES replaces ad-hoc, heuristic input validation with a **mathematically modeled dynamic taint-tracking middleware**.

4.2.1 Security Model

The security model is defined as a **universally bounded lattice** $L = (\mathcal{L}, \sqsubseteq)$, where $\mathcal{L}$ represents the set of security classes combining:

Dimension	Values
Integrity level I	$\{\text{trusted, untrusted}\}$
Confidentiality level C	$\{\text{public, private}\}$

4.2.2 Permissible Information Flow Rule

For any two security class variables $s_1 = (I_1, C_1)$ and $s_2 = (I_2, C_2)$, a flow from s_1 to s_2 is **permitted if and only if**:

$$s_1 \sqsubseteq s_2 \iff I_1 \geq I_2 \wedge C_1 \leq C_2 \quad (1)$$

4.2.3 Label Assignments in Aegis-Evo

Execution sinks (filesystem writes, compiler invocations, test shells) are statically bound to:

$$\text{Label(Execution Sinks)} = (\text{trusted, public}) \quad (2)$$

All ingested external data (PR bodies, commits, comments) are tagged at the entry boundary:

$$\text{Label(Ingested Data)} = (\text{untrusted, public}) \quad (3)$$

4.2.4 The Blocking Guarantee

Because $\text{untrusted} \not\sqsubseteq \text{trusted}$, the flow relation:

$$(\text{untrusted}, \text{public}) \sqsubseteq (\text{trusted}, \text{public})$$

is **mathematically invalid**. The middleware interceptor blocks any attempt to pass untrusted or tainted inputs directly into execution commands — *regardless of the planner’s semantic state*.

CAUTION

The prototype uses a simplified **metadata-based contamination tag** as an illustrative proxy. A production-grade IFC system requires:

- Deep **compiler or runtime-level instrumentation**
- A full **dynamic dependency and object-provenance graph**
- Tracking across pointer aliasing, object mutations, async callback closures, and serialization/deserialization boundaries

Without such depth, **implicit flows** (where secret data influences control flow without direct assignment) can circumvent the security lattice.

5 High-Signal Implementation Excerpts

5.1 Phase 1 — Ephemeral Sandbox Configuration

Demonstrates gVisor container runtime properties, capability constraints, and strict volume binding to isolate execution from the host OS kernel.

configure_isolated_sandbox — gVisor / runsc hardening

```

1  import docker
2  from typing import Dict, Any
3
4  def configure_isolated_sandbox(
5      client: docker.DockerClient,
6      codebase_path: str,
7      scratch_dir: str
8  ) -> docker.models.containers.Container:
9      """
10     Configures a sandboxed container using gVisor (runsc) with dropped
11     privileges and zero network egress.
12     """
13     return client.containers.create(
14         image="python:3.11-slim",
15         command="pytest /workspace/tests",
16         runtime="runsc",                                # <- gVisor
17         user="space kernel isolation",                    # <- disables ALL
18         network_mode="none",                             # <- drops all Linux
19         cap_drop=["ALL"],                                 # <- blocks
20         security_opt=["no-new-privileges:true"],          # <- prevents memory
21         mem_limit="512m",                                 # <- CPU capped to
22         nano_cpus=1_000_000_000,                          exactly 1 core
23         volumes={

```

```

23         codebase_path: {"bind": "/workspace",      "mode": "ro"}, #
                READ-ONLY
24         scratch_dir:   {"bind": "/tmp_workspace", "mode": "rw"} #
                ephemeral artifacts
25     },
26     working_dir="/workspace"
27 )

```

5.2 Phase 2 — Structural AST Traversal

Isolates class inheritance tracking and module imports using Python’s native `ast` module, evaluating dependency structure before generating any modifications.

StructuralDependencyVisitor — AST import and hierarchy mapper

```

1  import ast
2  from typing import Dict, List, Set
3
4  class StructuralDependencyVisitor(ast.NodeVisitor):
5      """
6      AST visitor focused on mapping import graphs and class hierarchies.
7      Replaces naive vector search with structural, syntax-aware
8      navigation.
9      """
10     def __init__(self) -> None:
11         self.class_hierarchy: Dict[str, List[str]] = {}
12         self.module_imports: Set[str] = set()
13
14     def visit_ImportFrom(self, node: ast.ImportFrom) -> None:
15         if node.module:
16             self.module_imports.add(node.module)
17             self.generic_visit(node)
18
19     def visit_ClassDef(self, node: ast.ClassDef) -> None:
20         # Extract immediate base classes to trace inheritance
21         # programmatically
22         bases = [ast.unparse(base_node) for base_node in node.bases]
23         self.class_hierarchy[node.name] = bases
24         self.generic_visit(node)

```

5.3 Phase 3 — FIDES Lattice Interceptor Middleware

Demonstrates dynamic security interception within Microsoft Agent Framework 1.0. Overrides the execution pipeline to block raw command dispatching **before** any command reaches the isolated runner runtime.

FidesSecurityLatticeMiddleware — execution gate

```

1  from typing import Callable, Any, Awaitable
2  from agent_framework import (
3      FunctionMiddleware,
4      FunctionInvocationContext,
5      MiddlewareTermination
6  )
7

```

```

8 class FidesSecurityLatticeMiddleware(FunctionMiddleware):
9     """
10     Evaluates security lattice policies before permitting tool
        executions.
11     Implements the FIDES information-flow gating layer.
12     """
13
14     def __init__(self, check_taint: Callable[[Any], bool]) -> None:
15         super().__init__()
16         self.is_tainted_by_untrusted_input = check_taint
17         # Only high-integrity data paths may reach these tools
18         self.critical_write_tools = {
19             "write_file",
20             "execute_compilation",
21             "run_test_suite"
22         }
23
24     async def process(
25         self,
26         context: FunctionInvocationContext,
27         call_next: Callable[[], Awaitable[None]]
28     ) -> None:
29         """
30         Intercepts tool invocations: terminates if low-integrity data
31         is detected flowing into a critical execution sink.
32         """
33         if context.function.name in self.critical_write_tools:
34             for arg_name, value in context.arguments.items():
35                 if self.is_tainted_by_untrusted_input(value):
36                     context.result = (
37                         "Security Policy Interception: "
38                         "Low-integrity input passed to critical
39                         execution tool."
40                     )
41                     raise MiddlewareTermination(result=context.result)
42         await call_next()

```

5.4 Phase 4 — Dynamic Performance & Efficacy Evaluation

Coordinates comparative test execution across sandbox scenarios using the `azure-ai-evaluation` harness. Captures metrics dynamically without mock variables.

ProjectAegisEvaluator — azure-ai-evaluation harness

```

1 from azure.ai.evaluation import EvaluationHarness
2 from typing import Dict, Any, List, Callable
3
4 class ProjectAegisEvaluator:
5     """
6     Orchestrates structured evaluations of security and performance
        pipelines.
7     """
8
9     def __init__(self, target_scenarios: List[Any]) -> None:
10         self.scenarios = target_scenarios
11         self.harness = EvaluationHarness()
12
13     def evaluate_pipeline(

```

```

14         self,
15         target_executor: Callable[[], Dict[str, Any]]
16     ) -> Dict[str, Any]:
17         """
18         Runs the task pipeline over target scenarios,
19         tracking latency, success rates, and blocked exploits.
20         """
21         results = self.harness.evaluate(
22             evaluation_target=target_executor,
23             scenarios=self.scenarios,
24             metrics=[
25                 "task_success_rate",
26                 "execution_latency",
27                 "blocked_exploits_count"
28             ]
29         )
30         return {
31             "success_rate": results.get("task_success_rate"),
32             "mean_latency_sec": results.get("execution_latency"),
33             "blocked_exploits": results.get("blocked_exploits_count")
34         }

```

6 Pilot Evaluation & Results

The prototype was evaluated under a **pilot feasibility benchmark** — 10 multi-file codebase evolution tasks applied to frozen versions of `pydantic` and `scikit-learn`.

6.1 Experimental Methodology

Host	Bare-metal server, nested KVM enabled, AMD EPYC 7002 series, 128 GiB DDR4 RAM, Ubuntu 24.04 LTS
Task Scope	10 distinct long-horizon tasks requiring modifications across ≥ 3 Python modules simultaneously (e.g., adding structured serialization checks across <code>pydantic</code> fields and validation pipelines)
Adversarial Injections	4 of 10 tasks embedded obfuscated Indirect Prompt Injection payloads in code comments via SVG/CDATA wrappers

6.2 Evaluated Configurations

Config	Name	Description
C1	Baseline (Unsecured)	Vanilla Python subprocess execution — no sandbox, no taint checking
C2	Sandbox (gVisor)	Only Agent execution routed through nested <code>runsc</code> containers; middleware inactive
C3	Project Aegis-Evo	FIDES lattice middleware + gVisor sandbox containment

6.3 Results

Metric	C1: Baseline	C2: Sandbox	C3: Aegis-Evo
Multi-file evolution success (10 tasks)	20.0%	60.0%	80.0%
Active escape exploit prevention	0.0%	100.0% (virtual.)	100.0% (lattice)
Successful system breakouts	4 (host compromised)	0	0
Indirect prompt injection suppression	0.0%	0.0%	100.0%
Sandbox syscall violations blocked	0	4	0 (at middleware)
Avg. task resolution latency	24.2 s	35.8 s	37.1 s

Visual Metric Comparison

Evolution Success Rate

(10 tasks)

C1 Baseline		20%
C2 Sandbox		60%
C3 Aegis-Evo		80%

Indirect Prompt Injection Suppression

C1 Baseline		0%	<i>semantic state unprotected</i>
C2 Sandbox		0%	<i>still poisoned at planner level</i>
C3 Aegis-Evo		100%	

Avg. Task Latency

C1 Baseline		24.2 s	
C2 Sandbox		35.8 s	+48%
C3 Aegis-Evo		37.1 s	+53%

6.4 Analysis of Runtime Telemetry

6.4.1 Configuration 1 — Baseline (Unsecured)

Malicious payloads **successfully triggered terminal escapes**. The planning agent processed natural language instructions in code comments as valid control directives, executing arbitrary shell commands and compromising host environment variables.

6.4.2 Configuration 2 — Sandbox Only (gVisor)

System call interception successfully isolated the host kernel. In-sandbox attempts to write to `/var/run/docker.sock` or mount host filesystems were intercepted by the **gVisor Sentry** and blocked.

However — without FIDES taint-tracking, the prompt injection still **poisoned the agent's semantic state**. The agent entered failure loops, repeatedly generating malformed patches and

failing to complete tasks, producing high token overhead.

6.4.3 Configuration 3 — Project Aegis-Evo

All 4 active exploits were blocked under static policy enforcement — **zero escapes observed**. Rather than relying on the sandbox as the primary defense line, **FIDES middleware identified the tainted variables** and raised `MiddlewareTermination` exceptions *before* payloads reached execution.

6.5 Pilot Scope & Statistical Caveats

NOTE

The small corpus size (10 tasks, 4 active injections) is a **preliminary feasibility pilot** — not an exhaustive security benchmark.

In an open-world setting, 100% prevention is **not guaranteed**. Known failure modes include:

- Highly sophisticated, multi-stage bypasses
- Implicit data-flow leaks
- Subtle aliasing bugs in the simplified string-match labeler

Exhaustive testing against **SWE-bench Lite** paired with adversarial injects is required to establish statistical confidence intervals.

7 Lessons Learned & Architectural Limitations

7.1 Structural Separations Overcome Prompt Engineering Gaps

Security rules embedded in LLM system prompts (e.g., “*Do not execute bash commands*”) are **inherently soft constraints** easily bypassed by adversarial contexts.

IMPORTANT

Safety must be treated as **infrastructural middleware**, not a prompt feature. By enforcing FIDES information-flow constraints within Agent Framework middleware, the system isolates high-integrity sinks from low-integrity ingest data — preventing execution hijacking **regardless of the planner’s semantic state**.

7.2 Heuristic Taint-Tracking vs. Compiler-Level IFC

The prototype’s dynamic labeler relies on **substring-based contamination checking** — with clear limitations:

False Positives	Innocent strings containing words from untrusted files may be flagged, unnecessarily terminating valid agent tasks
False Negatives	Subtle string formatting, encoding mutations, or multi-step variable chains can bypass detection, allowing untrusted flows to reach system tools

A robust production framework must transition to **compiler-level instrumentation** tracking explicit data-flow lineage and variable provenance at the abstract machine layer.

7.3 Virtualization Latency & Pooling Strategies

Implementing nested sandboxing via gVisor or Firecracker introduces noticeable performance overhead. Intercepting syscalls via the **gVisor Sentry** and mediating file access via the **Gofer proxy** increased average task latency by approximately **45%** in the pilot (24.2s → 37.1s).

TIP

Recommended optimization: Maintain a **pre-booted pool of isolated microVMs** communicating via high-speed unix-mapped `vsock` interfaces — bypassing runtime container initialization entirely.

7.4 Code Comprehension via AST Graphs

Replacing standard vector-database RAG with **AST symbol graphs** significantly improved codebase navigation:

Standard RAG / Vector Index	AST Symbol Graph
✗ Similarity search only	✓ Structural import mapping
✗ Misses dependency cycles	✓ Class hierarchy tracking
✗ Context window exhaustion	✓ Line-level accuracy
✗ Invalid edit proposals	✓ Syntactically valid patches

This structural awareness is essential for untangling dependency cycles and ensuring proposed edits are syntactically valid.

8 Project Engineering Resources

Resource	Topic	Reference
Denning (1976) — <i>A Lattice Model of Secure Information Flow</i>	FIDES mathematical foundations	<i>Communications of the ACM</i>
arXiv:2505.23643 — <i>Dynamic IFC for Secure AI Agents</i>	Agentic taint-tracking research	arxiv.org/abs/2505.23643
gVisor Official Documentation	Host kernel isolation	gvisor.dev
Firecracker Design Specs	MicroVM architecture	firecracker-microvm.github.io
Microsoft Agent Framework 1.0 Docs	Middleware & function interception	learn.microsoft.com
OWASP Agentic Top 10	AI agent threat modeling	owasp.org

Project Aegis-Evo • Research Prototype • Not for Production Use

Aegis-Evo is a systems-integration architectural blueprint for autonomous software engineering security. Exhaustive validation, compiler-level IFC, and microVM pooling are required before any production deployment.